

Laboratório 01 – Importação e Manipulação de Dados em R

Manipulação de Dados no Formato Tidy

Pedro Tilhe

2026-08-18

Introdução

Neste laboratório trabalhamos com os bancos de dados de casos de tuberculose (`table1` a `table4b`) distribuídos com o pacote `tidyverse`, com o objetivo de caracterizar o formato *tidy* e manipular os dados usando `dplyr`, `tidyr` e `ggplot2`.

1. Carregando o pacote `tidyverse`

```
library(tidyverse)
```

2. Os bancos `table1`, `table2`, `table3`, `table4a` e `table4b`

`table1`

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>      <int> <int>      <int>
## 1 Afghanistan 1999     745   19987071
## 2 Afghanistan 2000    2666   20595360
## 3 Brazil      1999   37737   172006362
## 4 Brazil      2000   80488   174504898
## 5 China       1999  212258  1272915272
## 6 China       2000  213766  1280428583
```

`table2`

```
## # A tibble: 12 x 4
##   country      year type          count
##   <chr>      <int> <chr>      <int>
## 1 Afghanistan 1999 cases          745
## 2 Afghanistan 1999 population 19987071
## 3 Afghanistan 2000 cases          2666
## 4 Afghanistan 2000 population 20595360
## 5 Brazil      1999 cases          37737
## 6 Brazil      1999 population 172006362
## 7 Brazil      2000 cases          80488
## 8 Brazil      2000 population 174504898
## 9 China       1999 cases          212258
## 10 China      1999 population 1272915272
## 11 China      2000 cases          213766
## 12 China      2000 population 1280428583
```

table3

```
## # A tibble: 6 x 3
##   country      year rate
## * <chr>      <int> <chr>
## 1 Afghanistan 1999 745/19987071
## 2 Afghanistan 2000 2666/20595360
## 3 Brazil      1999 37737/172006362
## 4 Brazil      2000 80488/174504898
## 5 China       1999 212258/1272915272
## 6 China       2000 213766/1280428583
```

table4a

```
## # A tibble: 3 x 3
##   country      `1999` `2000`
## * <chr>      <int> <int>
## 1 Afghanistan    745    2666
## 2 Brazil        37737   80488
## 3 China         212258  213766
```

table4b

```
## # A tibble: 3 x 3
##   country      `1999`      `2000`
## * <chr>      <int>      <int>
## 1 Afghanistan 19987071    20595360
## 2 Brazil      172006362   174504898
## 3 China       1272915272  1280428583
```

Um conjunto de dados é dito **tidy** (arrumado) quando:

1. cada variável forma uma coluna;
2. cada observação forma uma linha;
3. cada tipo de unidade observacional forma uma tabela.

Analisando cada banco:

- **table1:** está no formato *tidy*. Cada uma das quatro colunas (**country**, **year**, **cases**, **population**) corresponde a uma única variável, e cada linha representa uma observação (um país em um ano). Não há variáveis misturadas em uma mesma coluna nem uma mesma variável espalhada em várias colunas.
- **table2:** **não** está no formato *tidy*. A coluna **type** não é uma variável propriamente dita: ela armazena os *nomes* de duas variáveis diferentes ("**cases**" e "**population**"), e a coluna **count** armazena os valores dessas duas variáveis misturados. Ou seja, cada observação (país-ano) está espalhada em duas linhas.
- **table3:** **não** está no formato *tidy*. A coluna **rate** contém, em formato de texto ("**casos/população**"), duas variáveis (número de casos e tamanho da população) combinadas em uma única coluna. Duas variáveis deveriam ocupar duas colunas separadas, e não uma só.
- **table4a:** **não** está no formato *tidy*. Os nomes das colunas 1999 e 2000 não são variáveis, e sim *valores* da variável **year**. Além disso, os valores armazenados nessas colunas correspondem a apenas uma variável (**cases**), de modo que cada observação (país-ano) está espalhada em duas colunas em vez de estar em uma linha.
- **table4b:** pelo mesmo motivo de **table4a**, **não** está no formato *tidy*: os nomes das colunas 1999 e 2000 são valores de **year**, e os valores nas colunas correspondem à variável **population**.

3. Taxa de ocorrência de tuberculose por 10.000 pessoas

```
taxas <- table1 %>%
  mutate(rate = cases / population * 10000)

taxas

## # A tibble: 6 x 5
##   country      year  cases population  rate
##   <chr>      <int> <int>      <int> <dbl>
## 1 Afghanistan  1999     745   19987071 0.373
## 2 Afghanistan  2000    2666   20595360 1.29
## 3 Brazil       1999   37737  172006362 2.19
## 4 Brazil       2000   80488  174504898 4.61
## 5 China        1999  212258  1272915272 1.67
## 6 China        2000  213766  1280428583 1.67
```

4. Número de casos de tuberculose por ano

```
table1 %>%
  group_by(year) %>%
  summarise(casos = sum(cases))

## # A tibble: 2 x 2
##   year  casos
##   <int> <int>
## 1  1999 250740
## 2  2000 296920
```

5. Número de casos de tuberculose por país

```
table1 %>%
  group_by(country) %>%
  summarise(casos = sum(cases))

## # A tibble: 3 x 2
##   country      casos
##   <chr>      <int>
## 1 Afghanistan   3411
## 2 Brazil       118225
## 3 China        426024
```

6. Mudança no número de casos entre 1999 e 2000, por país

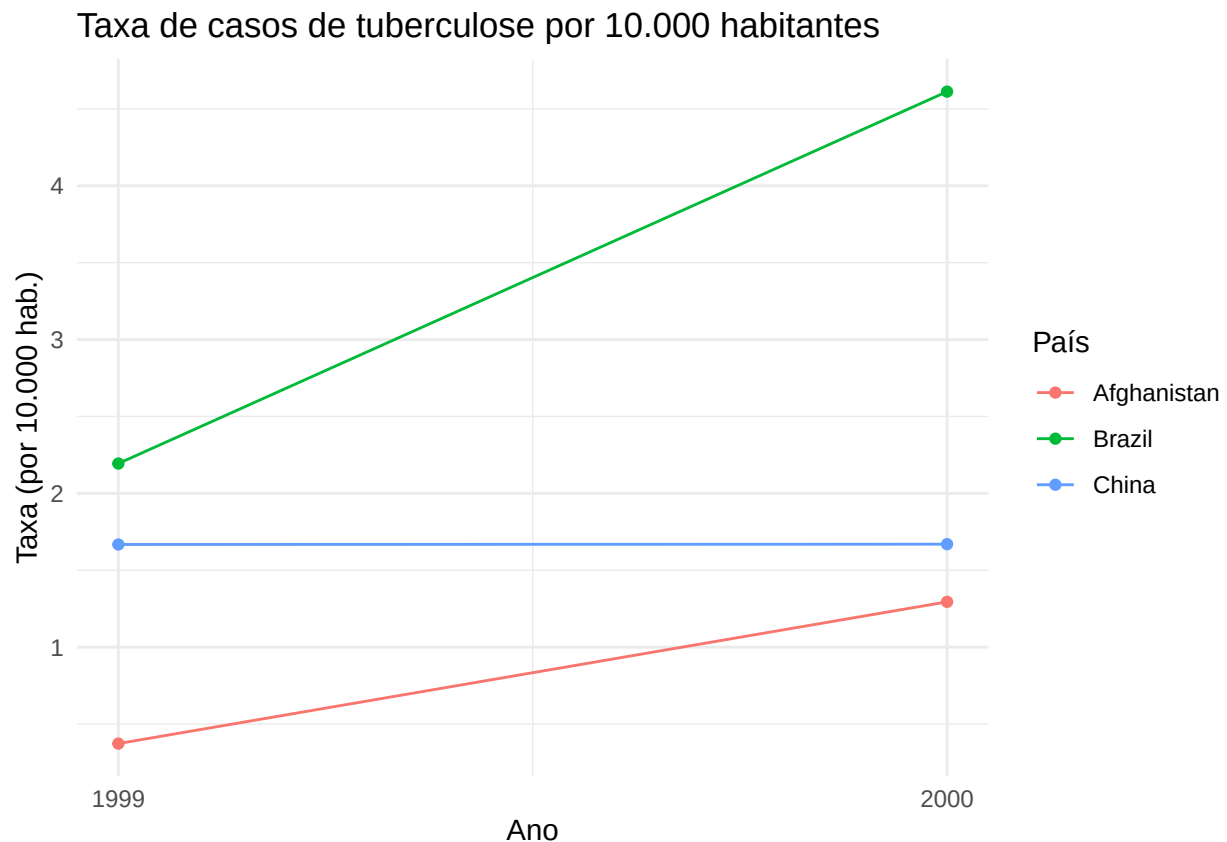
```
table1 %>%
  select(country, year, cases) %>%
  pivot_wider(names_from = year, values_from = cases) %>%
  mutate(mudanca = `2000` - `1999`)

## # A tibble: 3 x 4
##   country      `1999` `2000` mudanca
##   <chr>      <int> <int>   <int>
## 1 Afghanistan     745    2666    1921
## 2 Brazil       37737   80488   42751
```

3 China 212258 213766 1508

7. Gráfico de linhas: taxa de casos por país ao longo do tempo

```
ggplot(taxas, aes(x = year, y = rate, color = country, group = country)) +  
  geom_line() +  
  geom_point() +  
  scale_x_continuous(breaks = c(1999, 2000)) +  
  labs(  
    title = "Taxa de casos de tuberculose por 10.000 habitantes",  
    x = "Ano", y = "Taxa (por 10.000 hab.)", color = "País"  
  ) +  
  theme_minimal()
```



8. Taxa de casos para table2 e para table4a + table4b

Taxa a partir de table2:

```
# 1) número de casos por país/ano  
casos_table2 <- table2 %>%  
  filter(type == "cases") %>%  
  select(country, year, cases = count)  
  
# 2) população por país/ano  
pop_table2 <- table2 %>%  
  filter(type == "population") %>%
```

```
select(country, year, population = count)

# 3) e 4) junção, cálculo da taxa e armazenamento
taxa_table2 <- left_join(casos_table2, pop_table2, by = c("country", "year")) %>%
  mutate(rate = cases / population * 10000)
```

taxa_table2

```
## # A tibble: 6 x 5
##   country      year  cases population  rate
##   <chr>      <int> <int>      <int> <dbl>
## 1 Afghanistan 1999     745   19987071 0.373
## 2 Afghanistan 2000    2666   20595360 1.29
## 3 Brazil       1999   37737   172006362 2.19
## 4 Brazil       2000   80488   174504898 4.61
## 5 China        1999  212258  1272915272 1.67
## 6 China        2000  213766  1280428583 1.67
```

Taxa a partir de table4a (casos) e table4b (população):

```
# 1) número de casos por país/ano (table4a está "larga": um ano por coluna)
casos_table4 <- table4a %>%
  pivot_longer(cols = c(`1999`, `2000`), names_to = "year", values_to = "cases") %>%
  mutate(year = as.integer(year))
```

```
# 2) população por país/ano (table4b está "larga": um ano por coluna)
pop_table4 <- table4b %>%
  pivot_longer(cols = c(`1999`, `2000`), names_to = "year", values_to = "population") %>%
  mutate(year = as.integer(year))
```

```
# 3) e 4) junção, cálculo da taxa e armazenamento
taxa_table4 <- left_join(casos_table4, pop_table4, by = c("country", "year")) %>%
  mutate(rate = cases / population * 10000)
```

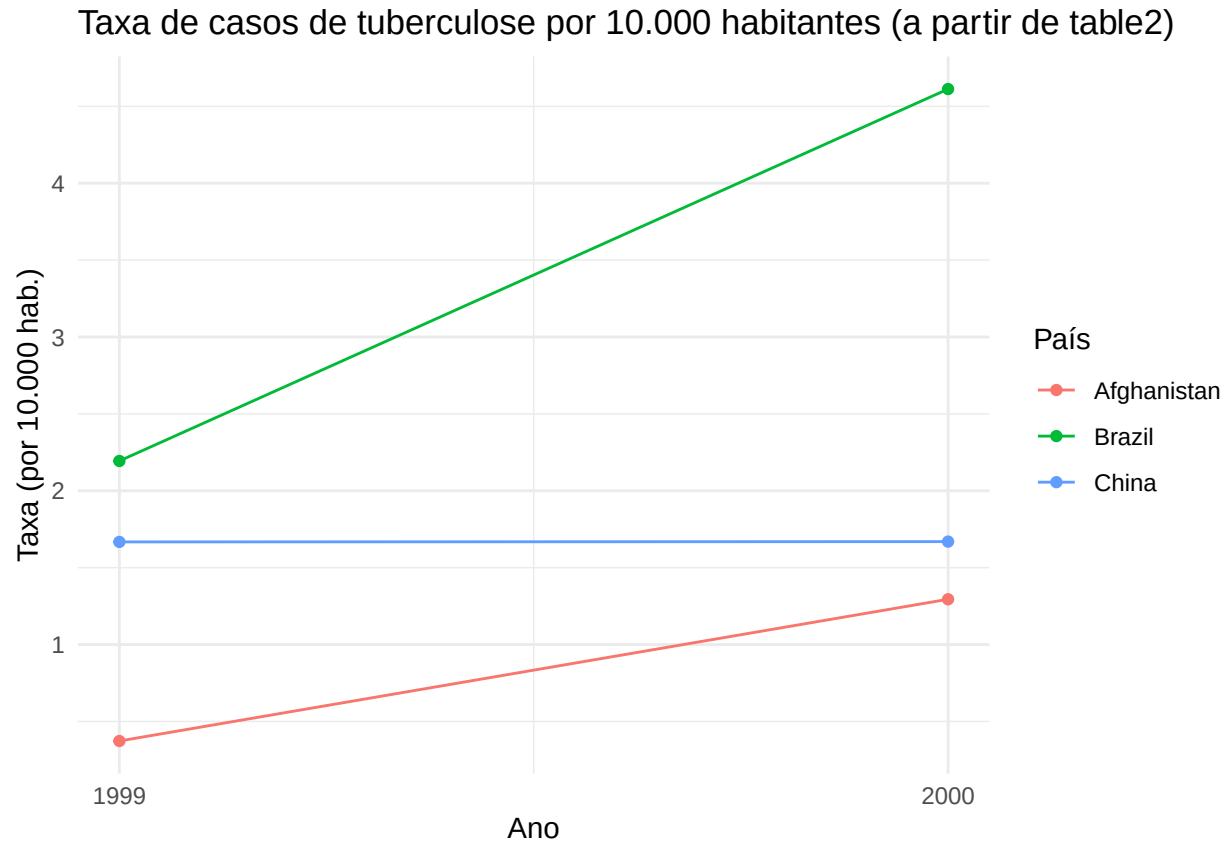
taxa_table4

```
## # A tibble: 6 x 5
##   country      year  cases population  rate
##   <chr>      <int> <int>      <int> <dbl>
## 1 Afghanistan 1999     745   19987071 0.373
## 2 Afghanistan 2000    2666   20595360 1.29
## 3 Brazil       1999   37737   172006362 2.19
## 4 Brazil       2000   80488   174504898 4.61
## 5 China        1999  212258  1272915272 1.67
## 6 China        2000  213766  1280428583 1.67
```

9. Gráfico da questão 7 feito com os dados de table2

```
ggplot(taxa_table2, aes(x = year, y = rate, color = country, group = country)) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = c(1999, 2000)) +
  labs(
    title = "Taxa de casos de tuberculose por 10.000 habitantes (a partir de table2)",
    x = "Ano", y = "Taxa (por 10.000 hab.)", color = "País"
```

```
) +  
theme_minimal()
```



Como esperado, o gráfico é idêntico ao da questão 7, já que `table1` e `table2` descrevem os mesmos dados, apenas organizados de formas diferentes.

10. Tornando `table4a` *tidy* com `pivot_longer`

```
tidy4a <- table4a %>%  
  pivot_longer(cols = c(`1999`, `2000`), names_to = "year", values_to = "cases") %>%  
  mutate(year = as.integer(year))
```

tidy4a

```
## # A tibble: 6 x 3  
##   country    year cases  
##   <chr>      <int> <int>  
## 1 Afghanistan 1999    745  
## 2 Afghanistan 2000   2666  
## 3 Brazil      1999  37737  
## 4 Brazil      2000  80488  
## 5 China       1999 212258  
## 6 China       2000 213766
```

11. Tornando table4b *tidy* com pivot_longer

```
tidy4b <- table4b %>%  
  pivot_longer(cols = c(`1999`, `2000`), names_to = "year", values_to = "population") %>%  
  mutate(year = as.integer(year))
```

tidy4b

```
## # A tibble: 6 x 3  
##   country      year population  
##   <chr>      <int>      <int>  
## 1 Afghanistan 1999    19987071  
## 2 Afghanistan 2000    20595360  
## 3 Brazil      1999    172006362  
## 4 Brazil      2000    174504898  
## 5 China       1999   1272915272  
## 6 China       2000   1280428583
```

12. Combinando tidy4a e tidy4b com left_join

```
tidy4 <- left_join(tidy4a, tidy4b, by = c("country", "year"))  
tidy4
```

```
## # A tibble: 6 x 4  
##   country      year cases population  
##   <chr>      <int> <int>      <int>  
## 1 Afghanistan 1999     745    19987071  
## 2 Afghanistan 2000    2666    20595360  
## 3 Brazil      1999   37737    172006362  
## 4 Brazil      2000   80488    174504898  
## 5 China       1999  212258   1272915272  
## 6 China       2000  213766   1280428583
```

O comando `left_join(x, y, by = ...)` combina as linhas de duas tabelas a partir de uma ou mais colunas-chave em comum (aqui, `country` e `year`). Todas as linhas da tabela à esquerda (`tidy4a`) são mantidas no resultado; para cada uma delas, o `left_join` procura na tabela à direita (`tidy4b`) uma linha com a mesma chave e, caso encontre, acrescenta as demais colunas dessa linha (aqui, `population`) ao resultado. Caso uma chave da tabela da esquerda não tenha correspondência na tabela da direita, as colunas vindas da direita ficam como NA. Neste caso, como `tidy4a` e `tidy4b` têm exatamente as mesmas combinações de `country` e `year`, o resultado é uma junção completa, reconstruindo uma tabela no mesmo formato de `table1`.

13. Tornando table2 *tidy* com pivot_wider

```
table2_tidy <- table2 %>%  
  pivot_wider(names_from = type, values_from = count)
```

table2_tidy

```
## # A tibble: 6 x 4  
##   country      year cases population  
##   <chr>      <int> <int>      <int>  
## 1 Afghanistan 1999     745    19987071  
## 2 Afghanistan 2000    2666    20595360  
## 3 Brazil      1999   37737    172006362
```

```
## 4 Brazil      2000  80488  174504898
## 5 China       1999 212258 1272915272
## 6 China       2000 213766 1280428583
```

O comando `pivot_wider` faz o processo inverso do `pivot_longer`: ele espalha os valores de uma coluna (`type`) em novas colunas, usando outra coluna (`count`) para preencher os valores. Como `type` continha os nomes das variáveis `cases` e `population`, essas passam a ser colunas de fato, e cada linha volta a representar uma única observação (país-ano) — exatamente o formato de `table1`.

14. Tornando `table3` *tidy* com `separate`

```
table3_tidy <- table3 %>%
  separate(rate, into = c("cases", "population"), sep = "/", convert = TRUE)
```

```
table3_tidy
```

```
## # A tibble: 6 x 4
##   country      year cases population
##   <chr>      <int> <int>      <int>
## 1 Afghanistan 1999     745   19987071
## 2 Afghanistan 2000    2666   20595360
## 3 Brazil      1999   37737   172006362
## 4 Brazil      2000   80488   174504898
## 5 China       1999  212258  1272915272
## 6 China       2000  213766  1280428583
```

O argumento `convert = TRUE` faz com que o `separate` tente automaticamente converter as novas colunas (`cases` e `population`), que seriam criadas como texto, para o tipo numérico mais apropriado (aqui, inteiro), evitando a necessidade de um passo extra de conversão manual com `as.numeric()`.